
SKILLFORGE: Co-Evolving Skills and Agents via Dynamic Skill Lifecycles

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Memory-augmented reinforcement learning strengthens LLM agents’ ability to
2 solve complex long-horizon tasks. Skills are one such form of memory, pairing
3 instructions with an applicability condition over task types. However, retaining
4 every skill indiscriminately as the policy improves lets obsolete or harmful entries
5 accumulate and mislead the agent. We propose **SKILLFORGE**, an agentic RL
6 method that compiles and evolves the skill library through a fitness-driven skill
7 lifecycle of `trial`, `active`, `stable`, and `retired` states, so that the skills and
8 the model co-evolve throughout training. A pre-RL evaluation phase first uses
9 the base model’s own rollouts to pre-retire low-fitness skills, yielding a filtered
10 library that then seeds supervised fine-tuning. Reinforcement learning takes over
11 from this checkpoint, and at each iteration selective retirement, stabilization, and
12 LLM-guided mutation continue to forge the skill library alongside policy optimiza-
13 tion. Across multiple interactive agent benchmarks, **SKILLFORGE** achieves the
14 highest aggregate success rate, delivering up to 7.8% relative improvement over
15 the strongest baseline while keeping the skill library compact throughout training.
16 We introduce **SKILLFURNACE**, a dataset of 5k+ annotated records bundling
17 retirement-filtered SFT trajectories, evolved skill libraries with fitness annotations,
18 and retirement events with human-annotated failure categories to support research
19 on skill quality and lifecycle management.

20 1 Introduction

21 Generalist agents such as Claude Code [Anthropic, 2025] and OpenClaw [OpenClaw, 2026] derive
22 much of their capability from modular, reusable abstractions provided at inference time. Even frontier
23 large language models (LLMs) sit behind a fixed knowledge cutoff, so they cannot absorb personalized
24 workflows [Chhikara et al., 2025] or adapt to post-deployment tooling conventions [Liu et al., 2025].
25 To close this gap at inference time, recent work centers on the *skill*: a bundle of instructions, examples,
26 and applicability conditions retrieved into the agent’s context on demand. Skill libraries are assembled
27 from procedural memory combining step-level and script-level abstractions [Fang et al., 2025], or
28 through autonomous skill-discovery loops [Yang et al., 2025]. As libraries grow, however, retrieval-
29 based augmentation degrades: top- k retrieval returns noisier context, outdated skills crowd out
30 high-quality ones, and many entries no longer match the agent’s current capability. Li et al. [2026a]
31 confirm these risks empirically: curated libraries contain skills that hurt performance, and larger
32 libraries yield diminishing returns.

33 A complementary line of work trains the policy via agentic reinforcement learning (Agentic RL)
34 against environment reward. Pairing skills or other accumulated memory with RL is a natural combi-
35 nation: skills focus exploration with behavioral priors, while RL shapes the policy through interaction.
36 Recent skill-augmented RL injects retrieved skills into the agent’s context each rollout [Xia et al.,
37 2026]; related threads close an offline-to-online distillation loop [Wu et al., 2025] or run policy

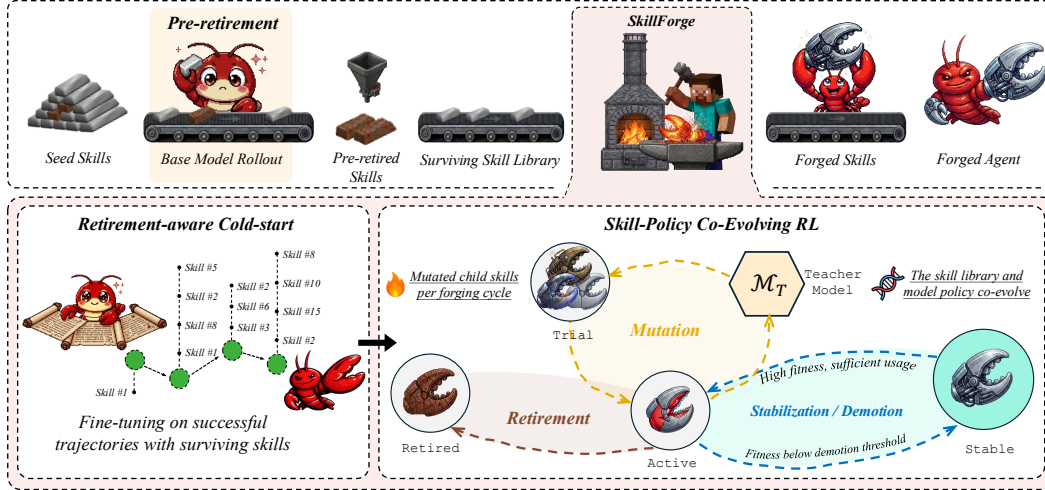


Figure 1: Overview of SKILLFORGE. **Top**: seed skills are pre-retired under the base model’s own rollouts; surviving fitness-verified skills seed retirement-aware cold-start and skill-policy co-evolving RL, yielding a trained agent with an adapted skill library. **Bottom-left**: cold-start fine-tunes on successful trajectories with surviving skills. **Bottom-right**: skills and model co-evolve through the fitness-driven lifecycle across trial, active, stable, and retired via selective retirement, stabilization, demotion, and LLM-guided mutation.

updates directly over episodic memory [Zhang et al., 2026]. Yet these methods all treat the library as append-only, leaving the agent’s context polluted by low-quality skills, bloated by unbounded accumulation, and misled by once-useful skills that turn counterproductive as the policy improves, a failure mode we call *delayed obsolescence*: a skill whose fitness once reached the stable state but later drops below the retirement threshold as the policy matures.

We propose SKILLFORGE, which treats skill library management as an active *forging* process rather than passive accumulation. Starting from a seed skill library distilled from trajectories, as illustrated in Figure 1, SKILLFORGE runs in three stages: (i) pre-retire seed skills that fail under the base model’s own rollouts, (ii) cold-start fine-tuning on the surviving fitness-verified skills, and (iii) switch to RL training where skills and model co-evolve each iteration. Each cycle reads runtime fitness and moves every skill along the skill lifecycle trial, active, stable, retired, with borderline-fitness skills rewritten by an LLM-guided mutation operator. Skills and model co-evolve, keeping the active set compact and aligned with the agent’s capability. We contribute the following:

- **SKILLFORGE and the skill lifecycle**: We propose SKILLFORGE, a fitness-driven skill lifecycle augmented with LLM-guided mutation that moves each skill through trial, active, stable, and retired states so that skills and model co-evolve, replacing the append-only accumulation of prior skill-augmented RL.
- **SKILLFURNACE dataset**: We release SKILLFURNACE, a three-part bundle of 5k+ annotated records spanning retirement-filtered SFT trajectories, evolved skill libraries with fitness annotations, and retirement events with human-annotated failure categories.
- **Empirical validation**: Across three interactive agent benchmarks spanning embodied control, web navigation, and Search-Augmented QA, SKILLFORGE surpasses the strongest baseline on every environment while keeping the library compact, delivering up to 7.8% relative improvement in aggregate success rate.

2 Related Work

Experience accumulation for LLM agents. Early work took the form of in-context reflection: self-critiquing the current output [Shinn et al., 2023, Madaan et al., 2023], and subsequently mining cross-trajectory insights [Majumder et al., 2023, Zhao et al., 2024]. In parallel, skill-library methods emerged: Wang et al. [2023] introduced an executable-code library for open-ended exploration,

67 followed by reusable task workflows [Wang et al., 2024], and more recently procedural memory
 68 mixing step-level instructions with script-level abstractions [Fang et al., 2025]. A related thread
 69 autonomously discovers new skills for the agent to practice [Zhou et al., 2024, Yang et al., 2025].
 70 On the RL side, memory-augmented methods couple such repositories to the training loop: earlier
 71 work used external memory to support tool-assisted planning [Putta et al., 2024, Bai et al., 2024],
 72 and later closed an offline-to-online loop between distillation and retrieval [Wu et al., 2025]. Most
 73 recently, Xia et al. [2026] combined skill libraries with RL training, injecting retrieved skills into the
 74 context during each rollout. In all of them the repository is monotone: entries persist once added
 75 and cannot be corrected as the policy evolves. SKILLFORGE instead treats the library as a dynamic
 76 population under fitness-driven selection, retiring low-fitness skills and reshaping borderline ones via
 77 LLM-guided mutation in step with the online RL optimizer.

78 **LLM-guided evolutionary optimization.** Jaderberg et al. [2017] introduced population-based
 79 training, applying fitness-based selection and mutation over the hyperparameter schedules of con-
 80 current workers. The same evaluate-select-mutate template has since been instantiated with LLMs:
 81 LLM-as-optimizer rewrites evolve a population of natural-language instructions against a fixed held-
 82 out metric [Yang et al., 2023, Guo et al., 2025], self-referential variants let instructions evolve each
 83 other [Fernando et al., 2023], and quality-diversity searches pair a pretrained LLM with an evaluator
 84 to discover diverse high-performing programs [Bradley et al., 2023, Romera-Paredes et al., 2023].
 85 These methods all optimize static text artifacts against a stationary held-out metric in an offline loop.
 86 SKILLFORGE brings fitness-driven selection and mutation into LLM agentic reinforcement learning,
 87 where the evolving entities are *skills* that augment the agent’s context during interactive rollouts.
 88 Joint online training makes skill fitness inherently non-stationary, requiring a lifecycle that retires and
 89 reshapes skills as the policy evolves.

90 3 Preliminaries

91 3.1 Group Relative Policy Optimization

92 GRPO [Shao et al., 2024] estimates advantages from grouped samples without a separate value func-
 93 tion. Given task description d drawn from a training task distribution $\mathcal{D}_{\text{task}}$, a uniform distribution over
 94 the training tasks of each environment, the old policy $\pi_{\theta_{\text{old}}}$ samples G episodes $\{\tau_n\}_{n=1}^G$ with binary
 95 rewards $r_n = R(\tau_n) \in \{0, 1\}$, and computes group-normalized advantages $\hat{A}_n = (r_n - \mu_G)/\sigma_G$.
 96 Let $\mathbf{y}_n = (y_{n,1}, \dots, y_{n,|\mathbf{y}_n|})$ denote the concatenation of action tokens produced autoregressively
 97 by the policy during episode τ_n ; agent contexts and observations serve as conditioning inputs rather
 98 than generation targets, and $\mathbf{y}_n$ aligns with the step-level actions $\{a_t\}$ of Section 3.2. Token-level
 99 importance ratios are $\rho_{n,l} = \pi_{\theta}(y_{n,l} | c_{n,l})/\pi_{\theta_{\text{old}}}(y_{n,l} | c_{n,l})$, where $c_{n,l}$ denotes the left context of
 100 token $y_{n,l}$, comprising all prior agent contexts, observations, and earlier action tokens of the episode.
 101 The index n indexes episodes within a GRPO group and is distinct from the skill index i used in
 102 Section 4. The policy is updated by maximizing:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{d \sim \mathcal{D}_{\text{task}}, \{\tau_n\}} \left[\frac{1}{G} \sum_{n=1}^G \frac{1}{|\mathbf{y}_n|} \sum_{l=1}^{|\mathbf{y}_n|} \min(\rho_{n,l} \hat{A}_n, \text{clip}(\rho_{n,l}, 1-\epsilon, 1+\epsilon) \hat{A}_n) \right] \quad (1)$$

$$- \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}),$$

103 with token-level clipping and a trajectory-level KL penalty against the reference policy.

104 3.2 Skill-Augmented RL

105 SkillRL [Xia et al., 2026] augments GRPO with a textual skill library $\mathcal{S}$, where each skill $s = (v, b, \omega)$
 106 comprises a name v , a behavioral principle b , and an applicability condition ω . Given task description
 107 d from Eq. (1), a retrieval function $\phi(d, \mathcal{S})$ selects the top- K skills whose condition ω matches the
 108 task-type label of d and concatenates them into the agent’s context:

$$p_t = d \oplus \phi(d, \mathcal{S}) \oplus h_t \oplus o_t, \quad (2)$$

109 where h_t is a sliding window of the last W observation–action pairs, o_t is the current observation,
 110 and $\oplus$ denotes string concatenation. At each step t , an action $a_t \sim \pi_{\theta}(\cdot | p_t)$ is sampled, and the

Table 1: SKILLFURNACE dataset statistics. The top block reports per-component record counts across SFT trajectories, library snapshots, and retirement events; the bottom block reports detailed retirement-event metrics. **Total records (#)** is summed across components within each environment.

Statistic	ALFWorld	WebShop	Search-Aug. QA	Total
<i>Dataset scale</i>				
SFT trajectories (#)	412	348	253	1,013
Library snapshot records (#)	1,512	1,683	1,326	4,521
Retirement events (#)	138	121	59	318
Total records (#)	2,062	2,152	1,638	5,852
<i>Retirement event statistics</i>				
Avg. peak fitness	0.696	0.709	0.515	0.667
Avg. fitness drop from peak	0.294	0.279	0.239	0.278
Avg. lifespan (steps)	38.8	31.7	49.4	38.0
Stable before retirement (%)	60.1	58.7	47.5	57.2

trajectory $\tau = (p_1, a_1, o_2, \dots, p_T, a_T)$ decomposes into context-action pairs $\{(p_t, a_t)\}_{t=1}^T$ used as SFT supervision units. Each episode receives a binary reward $R(\tau) \in \{0, 1\}$. Initially, skills are distilled from trajectories via OpenAI o3 [OpenAI, 2025], and a cold-start fine-tuning checkpoint $\pi_{\theta_{\text{sft}}}$ serves as both RL initialization and reference policy π_{ref} .

4 SKILLFORGE

As illustrated in Figure 1, SKILLFORGE runs in three stages: (i) pre-retirement evaluation filters low-fitness seed skills, (ii) retirement-aware cold-start fine-tuning on the surviving library, and (iii) agentic RL with a fitness-driven skill lifecycle where skills and policy co-evolve each iteration. This design addresses the non-stationarity of skill fitness under online RL: as the policy matures, once-useful skills may slip below the retirement threshold, *delayed obsolescence* formalized in Eq. (11).

4.1 SKILLFURNACE Initialization

We evaluate the initial skill library $\mathcal{S}_0$ released by SkillRL [Xia et al., 2026] with base model π_{θ_0} , which performs M rollout episodes with skills injected. For each $s_i \in \mathcal{S}_0$ we accumulate usage $U_i^{(0)}$ and success $C_i^{(0)}$. Skills whose proto-fitness falls below a conservative threshold δ_{pre} and that have been used at least N_{min} times are pre-retired:

$$\hat{f}_i = \frac{C_i^{(0)}}{U_i^{(0)}}, \quad \mathcal{S}_{\text{retire}}^{(0)} = \{s_i \in \mathcal{S}_0 \mid \hat{f}_i < \delta_{\text{pre}} \wedge U_i^{(0)} \geq N_{\text{min}}\}. \quad (3)$$

The surviving library $\mathcal{S}^{(0)} = \mathcal{S}_0 \setminus \mathcal{S}_{\text{retire}}^{(0)}$ seeds both the cold-start fine-tuning data and the RL phase described in Section 4.4. Let $\mathcal{T}_{\text{eval}}$ denote the trajectories from this evaluation, and $\phi_{\text{eval}}(d, \mathcal{S}_0)$ the retrieval ϕ from Eq. (2) on the full library $\mathcal{S}_0$ before pre-retirement. We construct the SFT dataset from successful trajectories:

$$\mathcal{D}_{\text{SFT}} = \{(p_t, a_t) \mid \tau \in \mathcal{T}_{\text{eval}}, (p_t, a_t) \in \tau, R(\tau) = 1, \phi_{\text{eval}}(d_\tau, \mathcal{S}_0) \cap \mathcal{S}_{\text{retire}}^{(0)} = \emptyset\}, \quad (4)$$

where d_τ is the task description of τ ; the filter excludes trajectories that relied on subsequently retired skills. Fine-tuning the base model on $\mathcal{D}_{\text{SFT}}$ via standard cross-entropy

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(p_t, a_t) \sim \mathcal{D}_{\text{SFT}}} [\log \pi_\theta(a_t \mid p_t)], \quad (5)$$

produces $\pi_{\theta_{\text{sft}}}$, which serves as both the RL initialization and the reference policy π_{ref} .

The filtered $\mathcal{D}_{\text{SFT}}$, the surviving library $\mathcal{S}^{(0)}$, and human-annotated retirement events from previously recorded training runs together form SKILLFURNACE, our released dataset. As summarized in Table 1, retired skills peak at an average fitness of 0.67, close to the stabilization threshold δ_{stable} of 0.7, before dropping by 0.28 at retirement, and 57.2% first reach the stable state before this decay: a direct empirical signature of *delayed obsolescence*. Appendix C gives per-component details.

4.2 Skill Fitness Tracking

For each skill s_i , we maintain a usage count U_i (episodes in which s_i was retrieved) and a success count C_i (episodes that succeeded). At the start of RL, the counters are initialized from the preliminary evaluation phase: $C_i \leftarrow C_i^{(0)}$ and $U_i \leftarrow U_i^{(0)}$ for each surviving skill $s_i \in \mathcal{S}^{(0)}$, so proto-fitness information from Section 4.1 is preserved rather than discarded. The fitness is:

$$f_i = \begin{cases} C_i/U_i & \text{if } U_i \geq N_{\text{warm}} \\ f_{\text{default}} & \text{otherwise} \end{cases} \quad (6)$$

where N_{warm} is a minimum usage count before trusting the empirical success rate and f_{default} is a neutral prior for new skills. We use $f_i^{(k)}$ for the running estimate at training step k when dependence on k matters, as formalized in Section 4.4; for any skill with $U_i^{(0)} \geq N_{\text{warm}}$ we have $f_i^{(0)} = \hat{f}_i$, so the proto-fitness from Eq. (3) is the runtime fitness at step 0, reused rather than recomputed.

Credit is assigned at the episode level: for each episode with trajectory τ and retrieved skill set $\mathcal{S}_\tau = \phi(d, \mathcal{S})$, all K retrieved skills receive the binary reward signal:

$$U_i \leftarrow U_i + \mathbb{I}[s_i \in \mathcal{S}_\tau], \quad C_i \leftarrow C_i + \mathbb{I}[s_i \in \mathcal{S}_\tau] \cdot R(\tau), \quad (7)$$

where $\mathbb{I}[\cdot]$ denotes the indicator function.

4.3 Skill Lifecycle Management

Based on accumulated fitness, we define a *lifecycle update rule* ℓ that assigns each skill one of four states $\{\text{trial}, \text{active}, \text{stable}, \text{retired}\}$; the active library $\mathcal{S}$ retains only non-retired skills. Transitions depend on fitness thresholds $\delta_{\text{retire}} < \delta_{\text{demote}} < \delta_{\text{stable}}$, where δ_{demote} lies strictly between the retirement and stabilization thresholds to provide hysteresis so that a stable skill whose fitness slips must first be demoted to active before becoming eligible for retirement. Transitions also depend on a mutation range $[\delta_{\text{low}}, \delta_{\text{high}}]$, minimum usage counts N_{promote} and N_{stable} , and a generation-aware retirement threshold:

$$N_{\text{retire}}^{(g_i)} = \begin{cases} N_{\text{protect}} & \text{if } g_i = 0 \\ N_{\text{retire}} & \text{otherwise} \end{cases} \quad (8)$$

where g_i is the mutation generation; distilled ($g_i = 0$) skills require a higher usage bar $N_{\text{protect}} > N_{\text{retire}}$ before they can be retired. The lifecycle update rule is then:

$$\ell(s_i) \leftarrow \begin{cases} \text{active} & \text{if } \ell(s_i) = \text{trial} \wedge U_i \geq N_{\text{promote}} \\ \text{stable} & \text{if } \ell(s_i) = \text{active} \wedge f_i \geq \delta_{\text{stable}} \wedge U_i \geq N_{\text{stable}} \\ \text{active} & \text{if } \ell(s_i) = \text{stable} \wedge f_i < \delta_{\text{demote}} \\ \text{retired} & \text{if } \ell(s_i) = \text{active} \wedge f_i < \delta_{\text{retire}} \wedge U_i \geq N_{\text{retire}}^{(g_i)} \end{cases} \quad (9)$$

If none applies, for example an active skill with $\delta_{\text{retire}} \leq f_i < \delta_{\text{stable}}$, $\ell(s_i)$ is left unchanged; Appendix B states the well-definedness and per-cycle ordering used by Algorithm 1. Only active skills are eligible for retirement; trial skills must first accumulate N_{promote} usages and promote to active, and stable skills must first be demoted. This two-gate design prevents premature retirement of newly created skills and shields proven skills from transient fitness dips. The mutation generation g_i is 0 for skills distilled into $\mathcal{S}_0$ and is incremented by 1 at each mutation.

Initial skills surviving pre-retirement enter as stable if $\hat{f}_i \geq \delta_{\text{stable}}$, or as active otherwise; newly generated skills enter as trial. The lifecycle operates through promotion, demotion, retirement, stabilization, and mutation, per Algorithm 1. Because demotion precedes retirement, a skill can be demoted and retired in one cycle, so once-stable skills can be retired as the policy evolves.

Mutation reshapes active skills whose fitness lies in a moderate range $[\delta_{\text{low}}, \delta_{\text{high}}]$ and which have been used at least N_{mutate} times, so that the fitness estimate used to select mutation parents is informative. From the resulting candidate pool $\mathcal{M}_{\text{cand}}$, we select up to K_{mutate} parents via inverse-fitness-weighted sampling ($P(s_i) \propto 1 - f_i$), collect their recent failure trajectories $\mathcal{T}_{\text{fail}}$, and query our teacher model ($\mathcal{M}_T$) to generate improved variants:

$$\begin{aligned} \mathcal{M}_{\text{cand}} &= \{s_i \in \mathcal{S} \mid \ell(s_i) = \text{active} \wedge \delta_{\text{low}} \leq f_i \leq \delta_{\text{high}} \wedge U_i \geq N_{\text{mutate}}\}, \\ s_{\text{child}} &= \mathcal{M}_T(\mathcal{P}_\mu(s_{\text{parent}}, f_{\text{parent}}, \mathcal{T}_{\text{fail}})), \end{aligned} \quad (10)$$

Algorithm 1 SKILLFORGE Forging Cycle

Require: Skill library $\mathcal{S}$, fitness scores $\{f_i\}$, usage counts $\{U_i\}$, failed trajectories $\mathcal{T}_{\text{fail}}$

Ensure: Updated skill library $\mathcal{S}$

```
1: for  $s_i \in \mathcal{S}$  where  $\ell(s_i) = \text{trial} \wedge U_i \geq N_{\text{promote}}$  do
2:    $\ell(s_i) \leftarrow \text{active}$  ▷ Promote: trial  $\rightarrow$  active
3: end for
4: for  $s_i \in \mathcal{S}$  where  $\ell(s_i) = \text{stable} \wedge f_i < \delta_{\text{demote}}$  do
5:    $\ell(s_i) \leftarrow \text{active}$  ▷ Demote: stable  $\rightarrow$  active
6: end for
7:  $\mathcal{R} \leftarrow \{s_i \in \mathcal{S} \mid \ell(s_i) = \text{active} \wedge f_i < \delta_{\text{retire}} \wedge U_i \geq N_{\text{retire}}^{(g_i)}\}$  ▷ Retirement candidates
8:  $\mathcal{S} \leftarrow \mathcal{S} \setminus \text{bottom-}m(\mathcal{R})$ ,  $m = \min(K_{\text{retire}}, |\mathcal{R}|)$  ▷ bottom- $m(\mathcal{R})$ :  $m$  skills in  $\mathcal{R}$  with lowest  $f_i$ 
9: for  $s_i \in \mathcal{S}$  where  $\ell(s_i) = \text{active} \wedge f_i \geq \delta_{\text{stable}} \wedge U_i \geq N_{\text{stable}}$  do
10:   $\ell(s_i) \leftarrow \text{stable}$  ▷ Stabilize: active  $\rightarrow$  stable
11: end for
12:  $\mathcal{M}_{\text{cand}} \leftarrow \{s_i \in \mathcal{S} \mid \ell(s_i) = \text{active} \wedge \delta_{\text{low}} \leq f_i \leq \delta_{\text{high}} \wedge U_i \geq N_{\text{mutate}}\}$  ▷ Mutation pool
13: Select  $K_{\text{mutate}}$  parents from  $\mathcal{M}_{\text{cand}}$ ;  $P(s_i) \propto 1 - f_i$  ▷ Lower fitness  $\rightarrow$  higher chance
14: for each selected  $s_{\text{parent}}$  do
15:   $s_{\text{child}} \leftarrow \mathcal{M}_T(\mathcal{P}_\mu(s_{\text{parent}}, f_{\text{parent}}, \mathcal{T}_{\text{fail}}))$  ▷ LLM-guided mutation
16:   $\mathcal{S} \leftarrow \mathcal{S} \cup \{s_{\text{child}}\}$ ;  $\ell(s_{\text{child}}) \leftarrow \text{trial}$ ;  $g_{\text{child}} \leftarrow g_{\text{parent}} + 1$  ▷ Child enters as trial
17: end for
18: while  $|\mathcal{S}| > S_{\text{max}}$  and  $\exists s_i \in \mathcal{S}$  with  $\ell(s_i) = \text{active}$  do
19:   $\mathcal{S} \leftarrow \mathcal{S} \setminus \{\arg \min_{s_i: \ell(s_i) = \text{active}} f_i\}$  ▷ Enforce size bound
20: end while
21: return  $\mathcal{S}$ 
```

where $\mathcal{P}_\mu$ assembles the parent skill’s content, fitness statistics, and failure examples into a structured template. Each child enters with $\ell(s_{\text{child}}) = \text{trial}$ and generation $g_{\text{child}} = g_{\text{parent}} + 1$. The inverse-fitness weighting $P(s_i) \propto 1 - f_i$ biases selection toward skills with more room to improve without concentrating mutation on a single outlier. We instantiate $\mathcal{M}_T$ from a different model family than the trained policy, since Li et al. [2026a] report that self-generated skills provide negligible or negative benefit compared with externally authored ones.

4.4 Training Integration

As given in Algorithm 1, a forging cycle runs every E validation epochs, with per-cycle budgets K_{retire} and K_{mutate} and a total skill cap S_{max} bounding computational cost.

Making the non-stationarity introduced at the start of Section 4 explicit, the fitness estimate at training step k depends on the policy trajectory:

$$f_i^{(k)} = \frac{\sum_{j \leq k} \mathbb{I}[s_i \in \mathcal{S}_{\tau^{(j)}}] \cdot R(\tau^{(j)})}{\sum_{j \leq k} \mathbb{I}[s_i \in \mathcal{S}_{\tau^{(j)}}]}, \quad \tau^{(j)} \sim \pi_{\theta_j}(\cdot \mid p^{(j)}), \quad (11)$$

where $p^{(j)}$ denotes the agent’s context at step j , constructed via Eq. (2) with the library $\mathcal{S}^{(j)}$. The $j = 0$ term covers the pre-retirement rollouts under π_{θ_0} , consistent with the counter initialization in Section 4.2 and yielding $f_i^{(0)} = \hat{f}_i$. As Li et al. [2026b] report, on-policy formulations avoid the exposure bias of off-policy alternatives, justifying the non-stationarity above. At each step k , the policy is updated via Eq. (1) conditioned on the current library, and the forging cycle updates the library based on accumulated fitness:

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{J}_{\text{GRPO}}(\theta; \mathcal{S}^{(k)}), \quad \mathcal{S}^{(k+1)} = \text{Forge}(\mathcal{S}^{(k)}, \{f_i^{(k)}\}), \quad (12)$$

where Forge denotes a full forging cycle. Skills and policy thus co-evolve: as θ improves, fitness estimates shift, and the forging cycle retires skills whose utility has diminished. *Delayed obsolescence* is then the formal event $\exists k_1 < k_2 : f_i^{(k_1)} \geq \delta_{\text{stable}} \wedge f_i^{(k_2)} < \delta_{\text{retire}}$.

Table 2: Performance on ALFWorld and WebShop (%). WebShop **Score** is the partial-credit reward; **Succ.** is binary task success. **Bold**: best; underline: second best.

Method	ALFWorld							WebShop	
	Pick	Look	Clean	Heat	Cool	Pick2	All	Score	Succ.
Closed-source LLMs									
GPT-4o	75.3	60.8	31.2	56.7	21.6	49.8	48.0	31.8	23.7
Gemini-2.5-Pro	92.8	63.3	62.1	69.0	26.6	58.7	60.3	42.5	35.9
Base Model									
Vanilla	33.4	21.6	19.3	6.9	2.8	3.2	14.8	26.4	7.8
Context / Memory Methods									
ReAct	48.5 _{▲15.1}	35.4 _{▲13.8}	34.3 _{▲15.0}	13.2 _{▲6.3}	18.2 _{▲15.4}	17.6 _{▲14.4}	31.2 _{▲16.4}	46.2 _{▲19.8}	19.5 _{▲11.7}
Reflexion	62.0 _{▲28.6}	41.6 _{▲20.0}	44.9 _{▲25.6}	30.9 _{▲24.0}	36.3 _{▲33.5}	23.8 _{▲20.6}	42.7 _{▲27.9}	58.1 _{▲31.7}	28.8 _{▲21.0}
Mem0	54.0 _{▲20.6}	55.0 _{▲33.4}	26.9 _{▲7.6}	36.4 _{▲29.5}	20.8 _{▲18.0}	7.7 _{▲4.5}	33.6 _{▲18.8}	23.9 _{▼2.5}	2.0 _{▼5.8}
ExpeL	21.0 _{▼12.4}	67.0 _{▲45.4}	55.0 _{▲35.7}	52.0 _{▲45.1}	71.0 _{▲68.2}	6.0 _{▲2.8}	46.3 _{▲31.5}	30.9 _{▲4.5}	11.2 _{▲3.4}
MemP	54.3 _{▲20.9}	38.5 _{▲16.9}	48.1 _{▲28.8}	56.2 _{▲49.3}	32.0 _{▲29.2}	16.7 _{▲13.5}	41.4 _{▲26.6}	25.3 _{▼1.1}	6.4 _{▼1.4}
SimpleMem	64.5 _{▲31.1}	33.3 _{▲11.7}	20.0 _{▲0.7}	12.5 _{▲5.6}	33.3 _{▲30.5}	3.8 _{▲0.6}	29.7 _{▲14.9}	33.2 _{▲6.8}	8.6 _{▲0.8}
RL-based Methods									
RLOO	87.6 _{▲54.2}	78.2 _{▲56.6}	87.3 _{▲68.0}	81.3 _{▲74.4}	71.9 _{▲69.1}	48.9 _{▲45.7}	75.5 _{▲60.7}	80.3 _{▲53.9}	65.7 _{▲57.9}
GRPO	90.8 _{▲57.4}	66.1 _{▲44.5}	89.3 _{▲70.0}	74.7 _{▲67.8}	72.5 _{▲69.7}	64.7 _{▲61.5}	77.6 _{▲62.8}	79.3 _{▲52.9}	66.1 _{▲58.3}
Memory-Augmented RL									
MemRL	62.8 _{▲29.4}	38.5 _{▲16.9}	22.2 _{▲2.9}	12.5 _{▲5.6}	8.0 _{▲5.2}	0.0 _{▼3.2}	21.4 _{▲6.6}	29.5 _{▲3.1}	9.2 _{▲1.4}
EvolveR	64.9 _{▲31.5}	33.3 _{▲11.7}	46.4 _{▲27.1}	13.3 _{▲6.4}	33.3 _{▲30.5}	33.3 _{▲30.1}	43.8 _{▲29.0}	42.5 _{▲16.1}	17.6 _{▲9.8}
Mem0+GRPO	78.1 _{▲44.7}	54.8 _{▲33.2}	56.1 _{▲36.8}	31.0 _{▲24.1}	65.0 _{▲62.2}	26.9 _{▲23.7}	54.7 _{▲39.9}	58.1 _{▲31.7}	37.5 _{▲29.7}
SimpleMem+GRPO	89.5 _{▲56.1}	36.3 _{▲14.7}	60.0 _{▲40.7}	50.0 _{▲43.1}	64.9 _{▲62.1}	26.3 _{▲23.1}	62.5 _{▲47.7}	67.8 _{▲41.4}	46.9 _{▲39.1}
SkillRL	97.9 _{▲64.5}	71.4 _{▲49.8}	90.0 _{▲70.7}	<u>90.0</u> _{▲83.1}	<u>95.5</u> _{▲92.7}	<u>87.5</u> _{▲84.3}	<u>89.9</u> _{▲75.1}	<u>85.2</u> _{▲58.8}	<u>72.7</u> _{▲64.9}
SKILLFORGE	98.2 _{▲64.8}	86.6 _{▲65.0}	<u>89.8</u> _{▲70.5}	93.7 _{▲86.8}	95.6 _{▲92.8}	88.4 _{▲85.2}	92.4 _{▲77.6}	87.8 _{▲61.4}	78.4 _{▲70.6}

5 Experiments

5.1 Experimental Setup

Environments. Three interactive agent benchmarks are used: ALFWorld [Shridhar et al., 2020], WebShop [Yao et al., 2022a], and Search-Augmented QA [Jin et al., 2025], with the Search-Augmented QA setting spanning NQ [Kwiatkowski et al., 2019], TriviaQA [Joshi et al., 2017], PopQA [Mallen et al., 2023], HotpotQA [Yang et al., 2018], 2WikiMultiHopQA [Ho et al., 2020], MuSiQue [Trivedi et al., 2022], and Bamboogle [Press et al., 2023].

Baselines. Baselines fall into three categories. **Context and memory methods** rely on in-context reasoning or external experience without updating the policy: ReAct [Yao et al., 2022b], Reflexion [Shinn et al., 2023], Mem0 [Chhikara et al., 2025], ExpeL [Zhao et al., 2024], MemP [Fang et al., 2025], and SimpleMem [Liu et al., 2025]. **RL-based methods** optimize the policy via group-relative advantage estimation: RLOO [Ahmadian et al., 2024] and GRPO [Shao et al., 2024]. **Memory-augmented RL methods** couple persistent memory with policy optimization and form the most direct comparison: MemRL [Zhang et al., 2026], EvolveR [Wu et al., 2025], SkillRL [Xia et al., 2026], and two additional combinations, Mem0+GRPO and SimpleMem+GRPO, implemented by adding GRPO optimization to the respective memory mechanisms. For Search-Augmented QA the comparison additionally includes Search-o1 [Li et al., 2025], R1-Instruct [Team, 2025a], Search-R1 [Jin et al., 2025], ZeroSearch [Sun et al., 2025], and StepSearch [Zheng et al., 2025]. GPT-4o [OpenAI, 2024] and Gemini-2.5-Pro [Team, 2025b] are reported as frontier reference points.

Implementation. All experiments are built on the *verl* [Sheng et al., 2025] framework with GRPO as the policy optimizer. The base model is Qwen2.5-7B-Instruct [Team, 2023], first fine-tuned via the retirement-aware SFT described in Section 4.1 to produce environment-specific initializations. Training runs for 200 steps with a learning rate of 1×10^{-6} , clipping range $\epsilon = 0.2$, and a KL penalty coefficient $\beta = 0.001$. Rollout generation uses temperature 1.0 with a group size of $G = 8$. The

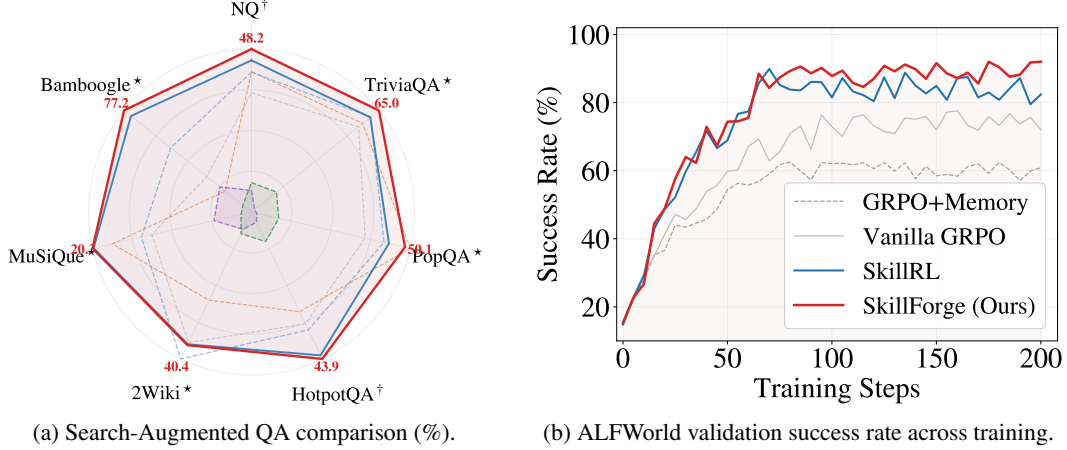


Figure 2: Main-results diagnostics. (a) Per-dataset accuracy on Search-Augmented QA; per-axis numbers in bold red report SKILLFORGE accuracy on each dataset. —: SKILLFORGE (Ours); —: SkillRL; - - : EvolveR; - - : ZeroSearch; - - : Search-R1; - - : Search-o1; - - : R1-Instruct. [†]In-domain; ^{*}out-of-domain. (b) SKILLFORGE achieves faster convergence and higher success rates than vanilla GRPO and memory-augmented RL.

Table 3: Ablation study of SKILLFORGE across three environments. “Library” reports the final non-retired skill count (coinciding with $|\mathcal{S}_0|$ for –forging/–mutation). Each row disables one forging stage: w/o forging removes the entire SKILLFORGE lifecycle, w/o pre-retirement keeps runtime stages but skips the pre-RL fitness filter, w/o retirement disables skill removal and size capping, and w/o mutation disables LLM-guided refinement. **Bold**: best.

Configuration	ALFWorld		WebShop		Search-Aug. QA	
	Library	Succ. (%)	Library	Succ. (%)	Library	Succ. (%)
SKILLFORGE	100	92.4	95	78.4	85	48.7
w/o forging	44	89.8 _{▼2.6}	66	72.6 _{▼5.8}	41	46.3 _{▼2.4}
w/o pre-retirement	100	90.7 _{▼1.7}	95	74.9 _{▼3.5}	85	47.5 _{▼1.2}
w/o retirement	132	91.4 _{▼1.0}	121	75.9 _{▼2.5}	111	47.9 _{▼0.8}
w/o mutation	44	91.8 _{▼0.6}	66	76.8 _{▼1.6}	41	48.2 _{▼0.5}

219 forging cycle executes every 10 training steps, with up to $K_{\text{mutate}} = 5$ mutations and $K_{\text{retire}} = 3$
220 retirements per cycle; the teacher model for mutation is Kimi-K2.5 [Team et al., 2026]. Full
221 hyperparameters, hardware details, and forging thresholds are listed in Appendix B.

222 5.2 Main Results

223 As shown in Table 2, SKILLFORGE achieves 92.4% on ALFWorld and 78.4% on WebShop, outper-
224 forming the closed-source GPT-4o and Gemini-2.5-Pro baselines by a wide margin and improving
225 over SkillRL by 2.8% relative on ALFWorld and 7.8% relative on WebShop. The largest per-subtask
226 relative improvement over SkillRL is on ALFWorld’s Look at 21.3%; the Clean subtask remains
227 within 0.2% of SkillRL, within sampling noise. On Search-Augmented QA, as illustrated in Figure 2a,
228 SKILLFORGE reaches $48.7 \pm 0.3\%$ micro-averaged accuracy on the full test set, the highest overall
229 among all methods; per-dataset results are in Appendix D.

230 As illustrated in Figure 2b, SKILLFORGE and SkillRL both climb from $\sim 60\%$ to $\sim 70\%$ between
231 steps 50 and 65, but SKILLFORGE separates beyond step 75 as the forging lifecycle takes effect,
232 reaching $\sim 90\%$ while SkillRL oscillates in the $\sim 80\text{--}88\%$ range over the rest of the training budget.

233 5.3 Ablation Study

234 As shown in Table 3, removing the entire SKILLFORGE lifecycle produces the largest relative drops
235 on every environment, peaking at 7.4% on WebShop. Pre-retirement is the single most impactful
236 component, with per-stage relative drops of 1.8%, 4.5%, and 2.5% on ALFWorld, WebShop, and

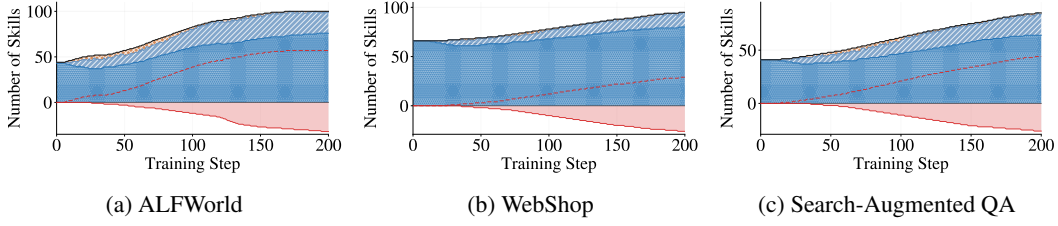


Figure 3: **Skill library dynamics** across three environments. **Stable**, **Active**, **Trial**: lifecycle states (stacked areas). **Retired**: cumulative retired (below zero). —: total non-retired; - -: mutated (gen $\geq$ 1).

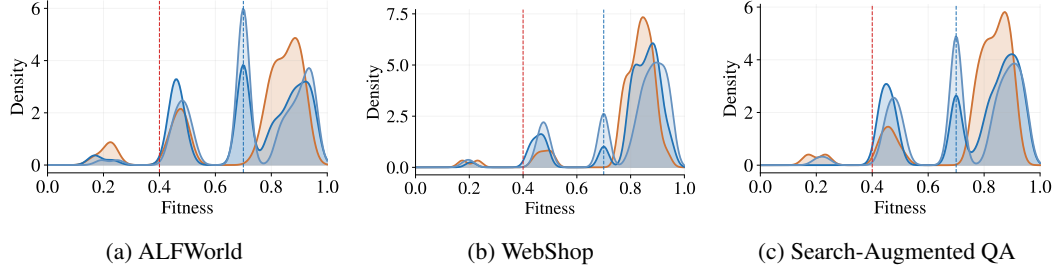


Figure 4: **Fitness distribution** at three training stages. **Step 40**, **Step 120**, **Step 195**: KDE at each stage. - -: $\delta_{\text{retire}} = 0.4$; - -: $\delta_{\text{stable}} = 0.7$. Retirement depletes the left tail, mutation populates the middle band via newly born skills, and stabilization maintains the high-fitness regime.

237 Search-Augmented QA respectively. The single-component drops consistently sum to more than the
 238 joint w/o forging drop, consistent with sub-additive interactions among the three components.

239 Larger libraries do not imply better performance. Without retirement the library grows to over 130
 240 skills on ALFWorld, yet SKILLFORGE outperforms it with a compact set of 100. On WebShop,
 241 enabling pre-retirement and mutation lifts the static 72.6% baseline to 75.9%; adding retirement
 242 trims the library from 121 skills to 95 and closes the remaining 3.2% relative gap to the full 78.4%,
 243 confirming that quality control, not scale, drives the gain.

244 5.4 Forging Dynamics

245 As shown in Figure 3, the skill library evolves similarly across the three environments. On ALFWorld,
 246 200 training steps grow the library to 100 non-retired skills, with 132 created and 32 retired; the cap
 247 binds near step 170, after which mutations and retirements balance into dynamic equilibrium. At the
 248 final checkpoint 76 skills are stable and 24 active or trial.

249 As illustrated in Figure 4, the distribution evolves through two opposing flows rather than a monotone
 250 rightward shift. Retirement depletes the left tail: on ALFWorld the sub- δ_{retire} fraction shrinks from
 251 7.5% to 2.0% across training, while mutation expands the middle band [$\delta_{\text{retire}}, \delta_{\text{stable}}$) from 18.9%
 252 to 22.0% as children enter with a neutral prior at 0.5. The stable regime retains most of the library
 253 throughout and mean fitness stays near 0.73, an equilibrium in which pruning and churn rebalance.
 254 Appendix E provides concrete mutation examples and the deepest forging lineages.

255 6 Conclusion

256 SKILLFORGE treats skill-library management as a fitness-driven forging process in which skills and
 257 the model co-evolve under online RL through retirement, stabilization, and LLM-guided mutation,
 258 delivering up to 7.8% relative improvement over the strongest memory-augmented RL baseline
 259 across ALFWorld, WebShop, and Search-Augmented QA while keeping the active library compact.
 260 We release SKILLFURNACE, a 5k+ annotated record bundle of retirement-filtered SFT trajectories,
 261 evolved skill libraries with fitness annotations, and retirement events with human-annotated failure
 262 categories, in which procedural rigidity emerges as the dominant mode by which once-useful skills
 263 turn into active interference, the empirical signature of *delayed obsolescence*.

References

- Anthropic. Claude Code. <https://claude.ai/>, 2025.
- OpenClaw. OpenClaw. <https://github.com/openclaw/openclaw>, 2026.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Jiaqi Liu, Yaofeng Su, Peng Xia, Yiyang Zhou, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2025. URL <https://github.com/aiming-lab/SimpleMem>.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *ArXiv*, abs/2508.06433, 2025.
- Yongjin Yang, Sinjae Kang, Juyong Lee, Dongjun Lee, Se young Yun, and Kimin Lee. Automated skill discovery for language agents through exploration and iterative feedback. *ArXiv*, abs/2506.04287, 2025.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026a. URL <https://arxiv.org/abs/2602.12670>.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, et al. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, et al. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *ArXiv*, abs/2601.03192, 2026.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *ArXiv*, abs/2303.17651, 2023.
- Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. Clin: A continually learning language agent for rapid task adaptation and generalization. *arXiv preprint arXiv:2310.10134*, 2023.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. 2024.
- Yifei Zhou, Qianlan Yang, Kaixiang Lin, Min Bai, Xiong Zhou, Yu-Xiong Wang, Sergey Levine, and Erran L. Li. Proposer-agent-evaluator(pae): Autonomous skill discovery for foundation model internet agents. *ArXiv*, abs/2412.13194, 2024.

- 310 Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and
311 Rafael Rafailov. Agent q: Advanced reasoning and learning for autonomous ai agents. *arXiv*
312 *preprint arXiv:2408.07199*, 2024.
- 313 Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. Digirl:
314 Training in-the-wild device-control agents with autonomous reinforcement learning. *Advances in*
315 *Neural Information Processing Systems*, 37:12461–12495, 2024.
- 316 Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M Czarnecki, Jeff Donahue, Ali
317 Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, et al. Population based training
318 of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.
- 319 Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun
320 Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning*
321 *Representations*, 2023.
- 322 Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and
323 Yujiu Yang. Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt
324 optimizers, 2025. URL <https://arxiv.org/abs/2309.08532>.
- 325 Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rock-
326 täschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint*
327 *arXiv:2309.16797*, 2023.
- 328 Herbie Bradley, Andrew Dai, Hannah Teufel, Jenny Zhang, Koen Oostermeijer, Marco Bellagente,
329 Jeff Clune, Kenneth Stanley, Grégory Schott, and Joel Lehman. Quality-diversity through ai
330 feedback. *arXiv preprint arXiv:2310.13032*, 2023.
- 331 Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog,
332 M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang,
333 Omar Fawzi, et al. Mathematical discoveries from program search with large language models.
334 *Nature*, 625:468 – 475, 2023.
- 335 Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang,
336 Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathemat-
337 ical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- 338 OpenAI. Introducing o3 and o4-mini. [https://openai.com/index/
339 introducing-o3-and-o4-mini/](https://openai.com/index/introducing-o3-and-o4-mini/), 2025.
- 340 Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-
341 ang Gao, Wenkai Yang, Zhiyuan Liu, et al. Rethinking on-policy distillation of large language
342 models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026b.
- 343 Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew
344 Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv*
345 *preprint arXiv:2010.03768*, 2020.
- 346 Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable
347 real-world web interaction with grounded language agents. *Advances in Neural Information*
348 *Processing Systems*, 35:20744–20757, 2022a.
- 349 Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and
350 Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement
351 learning. *arXiv preprint arXiv:2503.09516*, 2025.
- 352 Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris
353 Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a
354 benchmark for question answering research. *Transactions of the Association for Computational*
355 *Linguistics*, 7:453–466, 2019.
- 356 Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly
357 supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual*
358 *Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–
359 1611, 2017.

- 360 Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi.
361 When not to trust language models: Investigating effectiveness of parametric and non-parametric
362 memories. In *Proceedings of the 61st annual meeting of the association for computational*
363 *linguistics (volume 1: Long papers)*, pages 9802–9822, 2023.
- 364 Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov,
365 and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question
366 answering. In *Proceedings of the 2018 conference on empirical methods in natural language*
367 *processing*, pages 2369–2380, 2018.
- 368 Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-
369 hop QA dataset for comprehensive evaluation of reasoning steps. In Donia Scott, Nuria Bel,
370 and Chengqing Zong, editors, *Proceedings of the 28th International Conference on Compu-*
371 *tational Linguistics*, pages 6609–6625, Barcelona, Spain (Online), December 2020. Interna-
372 tional Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.580. URL
373 <https://aclanthology.org/2020.coling-main.580/>.
- 374 Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop
375 questions via single-hop question composition. *Transactions of the Association for Computational*
376 *Linguistics*, 10:539–554, 2022.
- 377 Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah Smith, and Mike Lewis. Measuring and
378 narrowing the compositionality gap in language models. In Houda Bouamor, Juan Pino, and Kalika
379 Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–
380 5711, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/
381 2023.findings-emnlp.378. URL <https://aclanthology.org/2023.findings-emnlp.378/>.
- 382 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan
383 Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international*
384 *conference on learning representations*, 2022b.
- 385 Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin,
386 Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for
387 learning from human feedback in LLMs. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar,
388 editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*
389 *(Volume 1: Long Papers)*, pages 12248–12267, Bangkok, Thailand, August 2024. Association for
390 Computational Linguistics. doi: 10.18653/v1/2024.acl-long.662. URL <https://aclanthology.org/2024.acl-long.662/>.
- 392 Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and
393 Zhicheng Dou. Search-ol: Agentic search-enhanced large reasoning models. In *Proceedings of the*
394 *2025 Conference on Empirical Methods in Natural Language Processing*, pages 5420–5438, 2025.
- 395 DeepSeek Team. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning.
396 *ArXiv*, abs/2501.12948, 2025a.
- 397 Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Yan Zhang,
398 Fei Huang, and Jingren Zhou. Zeroscore: Incentivize the search capability of llms without
399 searching. *arXiv preprint arXiv:2505.04588*, 2025.
- 400 Xuhui Zheng, Kang An, Ziliang Wang, Yuhang Wang, and Yichao Wu. StepSearch: Igniting
401 LLMs search ability via step-wise proximal policy optimization. In Christos Christodoulopoulos,
402 Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference*
403 *on Empirical Methods in Natural Language Processing*, pages 21805–21830, Suzhou, China,
404 November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi:
405 10.18653/v1/2025.emnlp-main.1106. URL [https://aclanthology.org/2025.emnlp-main.](https://aclanthology.org/2025.emnlp-main.1106/)
406 [1106/](https://aclanthology.org/2025.emnlp-main.1106/).
- 407 OpenAI. Gpt-4o system card, 2024. <https://openai.com/index/gpt-4o-system-card/>.
- 408 Gemini Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context,
409 and next generation agentic capabilities. *ArXiv*, abs/2507.06261, 2025b.

- 410 Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng,
411 Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of*
412 *the Twentieth European Conference on Computer Systems*, EuroSys '25, page 1279–1297. ACM,
413 March 2025. doi: 10.1145/3689031.3696075. URL [http://dx.doi.org/10.1145/3689031.](http://dx.doi.org/10.1145/3689031.3696075)
414 [3696075](http://dx.doi.org/10.1145/3689031.3696075).
- 415 Qwen Team. Qwen technical report. *ArXiv*, abs/2309.16609, 2023.
- 416 Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che,
417 Cheng Chen, Guanduo Chen, et al. Kimi k2.5: Visual agentic intelligence, 2026. URL [https:](https://arxiv.org/abs/2602.02276)
418 [//arxiv.org/abs/2602.02276](https://arxiv.org/abs/2602.02276).

419 A Notation

420 Table 4 summarizes the symbols used throughout the paper, grouped by domain.

Table 4: Notation used throughout the paper.

Symbol	Definition	Description
<i>Environment and Agent</i>		
d	Task description	Query input to the agent; d_τ denotes the instance tied to trajectory τ
$\mathcal{D}_{\text{task}}$	Training task distribution	Uniform over per-environment training tasks
R	Reward function	$R(\tau) \in \{0, 1\}$, binary task success
π_θ	Agent policy	Parameterized by θ
π_{θ_0}	Base model policy	Before any fine-tuning
$\pi_{\theta_{\text{old}}}$	Old policy (GRPO)	Generates rollouts; updated each training step
$\pi_{\theta_{\text{ft}}}$	SFT checkpoint policy	RL initialization and reference policy π_{ref}
$\tau = (p_1, a_1, o_2, \dots)$	Episode trajectory	Contexts, actions, observations over one episode
a_t	Agent action at step t	Sampled as $a_t \sim \pi_\theta(\cdot p_t)$
o_t	Observation at step t	Current environment observation
p_t	Agent context at step t	Built via Eq. (2); $p^{(j)}$ uses library $\mathcal{S}^{(j)}$ at training step j
h_t	Interaction history	Sliding window of the last $W=5$ (o, a) pairs
<i>Skill Library</i>		
$s_i = (v, b, \omega)$	Skill	Name, behavioral principle, applicability condition
$\mathcal{S}$	Skill library	Set of all non-retired skills
$\mathcal{S}_0$	Initial skill library	Input to Eq. (3); $\mathcal{S}_{\text{retire}}^{(0)}$ is the pre-retired subset
$\mathcal{S}^{(k)}$	Library at step k	After k forging updates; $\mathcal{S}^{(0)} = \mathcal{S}_0 \setminus \mathcal{S}_{\text{retire}}^{(0)}$
K	Retrieval count	Number of skills retrieved per episode
$\phi(d, \mathcal{S})$	Retrieval function	Selects top- K skills for task d ; ϕ_{eval} is the pre-retirement variant
$\mathcal{S}_\tau$	Retrieved skill set	$\phi(d, \mathcal{S})$ for trajectory τ
U_i	Usage count	Cumulative retrievals of s_i ; superscript (0) marks pre-retirement value
C_i	Success count	Cumulative episodes in which s_i was retrieved and succeeded
f_i	Skill fitness	C_i/U_i when $U_i \geq N_{\text{warm}}$, else f_{default}
f_{default}	Default fitness	Neutral prior for skills with $U_i < N_{\text{warm}}$
$\hat{f}_i$	Proto-fitness	$C_i^{(0)}/U_i^{(0)}$ from pre-retirement rollouts; equals $f_i^{(0)}$
$f_i^{(k)}$	Fitness at step k	Non-stationary estimate (Eq. (11))
$\ell(s_i)$	Lifecycle state	$\in \{\text{trial, active, stable, retired}\}$
g_i	Generation	Mutation depth; $g_i = 0$ for distilled skills
$\mathcal{M}_{\text{cand}}$	Mutation candidate pool	Active skills with moderate fitness (Eq. (10))
$\mathcal{M}_T$	Teacher model	Generates mutations via $\mathcal{P}_\mu$
$\mathcal{P}_\mu$	Mutation template	Assembles parent skill, fitness, and failures
$\mathcal{T}_{\text{fail}}$	Failure trajectories	Recent failed episodes used as mutation context
$\mathcal{D}_{\text{SFT}}$	SFT dataset	Successful trajectories filtered by retirement (Eq. (4))
$\text{Forge}(\cdot)$	Forging operator	Full forging cycle (Algorithm 1)
<i>Hyperparameters</i>		
M	Preliminary eval episodes	Rollout count for pre-retirement
δ_{pre}	Pre-retirement threshold	Proto-fitness below which initial skills are discarded
δ_{retire}	Retirement threshold	Fitness boundary below which active skills are retired
δ_{stable}	Stabilization threshold	Fitness boundary for active $\rightarrow$ stable
δ_{demote}	Demotion threshold	Fitness boundary for stable $\rightarrow$ active
$[\delta_{\text{low}}, \delta_{\text{high}}]$	Mutation fitness range	Bounds for mutation eligibility
N_{min}	Proto-fitness min usage	Min usage before proto-fitness is evaluated
N_{warm}	Fitness warm-up	Min usage before fitness replaces f_{default}
N_{promote}	Promotion min usage	Min usage for trial $\rightarrow$ active
N_{stable}	Stabilization min usage	Min usage for active $\rightarrow$ stable
N_{mutate}	Mutation min usage	Min usage for mutation eligibility
N_{retire}	Retirement min usage	Min usage before an active skill can be retired
$N_{\text{retire}}^{(g_i)}$	Generation-aware retirement	N_{protect} if $g_i = 0$, else N_{retire}
K_{protect}	Protection threshold	Higher usage bar for generation-0 skills
K_{retire}	Retirement budget	Max retirements per forging cycle
K_{mutate}	Mutation budget	Max mutations per forging cycle
S_{max}	Skill cap	Maximum library size
E	Forging period	Validation epochs between forging cycles
W	History window size	Recent (o, a) pairs kept in h_t ; $W=5$
<i>GRPO Training</i>		
$\mathcal{L}_{\text{SFT}}$	SFT loss	Cross-entropy on retirement-filtered trajectories (Eq. (5))
G	Group size	Trajectories sampled per task
$\hat{A}_n$	Group-normalized advantage	$(r_n - \mu_G)/\sigma_G$
ϵ	Clipping range	PPO-style surrogate clipping
β	KL coefficient	Regularization against reference policy
τ_n	Episode n in a GRPO group	Used for $\hat{A}_n$ and reward $r_n = R(\tau_n)$
$\mathbf{y}_n$	Token sequence of τ_n	Action tokens produced autoregressively by π_θ
$\tau^{(j)}$	Trajectory at step j	Rollout from π_{θ_j} for the non-stationary fitness (Eq. (11))
$y_{n,l}$	Token at position l	l -th action token in $\mathbf{y}_n$
$c_{n,l}$	Left context of $y_{n,l}$	Environment-step context plus earlier tokens
$\rho_{n,l}$	Importance ratio	$\pi_\theta(y_{n,l} c_{n,l})/\pi_{\theta_{\text{old}}}(y_{n,l} c_{n,l})$

B Implementation Details

Training configuration. Table 5 lists the training hyperparameters and hardware setup. The base model is Qwen2.5-7B-Instruct [Team, 2023], fine-tuned with GRPO on $64 \times$ NVIDIA H200 GPUs using the verl framework. The initial skill library is environment-specific: ALFWorld uses 44 skills (12 general, 21 task-specific, 11 common-mistake), WebShop uses 66 skills (15 general, 39 task-specific, 12 common-mistake), and Search-Augmented QA uses 41 skills (10 general, 20 query-type, 11 common-mistake), all inherited from the SkillRL release [Xia et al., 2026]. General skills encode cross-task heuristics (e.g., “always verify inventory after pickup”), task-specific skills target individual task types, and common-mistake skills capture frequently observed failure patterns from initial rollouts (e.g., “do not leave an object behind after finding it”).

Evaluation sample sizes. ALFWorld reports per-sub-task accuracy on the 130-task unseen-test set, partitioned as Pick (24), Look (15), Clean (27), Heat (26), Cool (21), and Pick2 (17); the **All** column in Table 2 is the sample-weighted micro-average across these. WebShop uses the standard 500-task held-out test set for both Score and Succ. Search-Augmented QA per-sub-dataset sample sizes appear in Table 10.

Table 5: Training configuration for SKILLFORGE. All three environments share the GRPO optimizer, group size, and training budget; hardware is identical across runs.

Hyperparameter	Value
Base model	Qwen2.5-7B-Instruct
GRPO learning rate	1×10^{-6}
Batch size	16
Group size G	8
Clipping range ϵ	0.2
KL coefficient β	0.001
Training steps	200
History window size W	5
Sampling temperature (train & eval)	1.0
Validation epoch	5 steps
Hardware	$64 \times$ NVIDIA H200
Framework	verl

Forging hyperparameters. Table 6 lists all forging-specific hyperparameters. The pre-retirement threshold $\delta_{\text{pre}} = 0.3$ is looser than the runtime $\delta_{\text{retire}} = 0.4$ because the pre-retirement phase uses the unadapted base policy π_{θ_0} , whose success rates are systematically lower than those observed during RL; a lower bar avoids discarding borderline-but-potentially-useful skills before RL has had a chance to re-evaluate them. Two usage-count thresholds also deserve clarification. The proto-fitness minimum $N_{\min} = 3$ (Eq. (3)) is smaller than the runtime warm-up $N_{\text{warm}} = 5$ (Eq. (6)) because proto-fitness is estimated under the fixed base policy over M dedicated rollouts, so three observations already yield a low-variance estimate for the binary filter $\hat{f}_i < \delta_{\text{pre}}$. Runtime fitness, in contrast, is estimated under a moving policy and feeds into continuous lifecycle transitions (promotion, stabilization, demotion, retirement), which justifies a slightly more conservative warm-up. Correspondingly, Eq. (3) is a hard *filter*: skills without $U_i^{(0)} \geq N_{\min}$ are simply not candidates for removal and remain in $\mathcal{S}^{(0)}$. In contrast, Eq. (6) must emit a numeric score for retrieval and ranking, so it falls back to the neutral prior $f_{\text{default}} = 0.5$ when the usage count is too low to trust the empirical rate. A second boundary case arises because $\delta_{\text{low}} = \delta_{\text{retire}} = 0.4$: a skill with f_i exactly equal to this value satisfies both the retirement condition (strict inequality $f_i < \delta_{\text{retire}}$ is false, so retirement does not fire) and the mutation range (closed interval $[\delta_{\text{low}}, \delta_{\text{high}}]$ includes the endpoint), so such a skill is eligible for mutation but not retirement and behaves identically to skills with f_i slightly above 0.4.

Lifecycle well-definedness. The four branches of Eq. (9) have pairwise disjoint guards: they require $\ell(s_i) \in \{\text{trial}, \text{active}, \text{stable}, \text{active}\}$ respectively, and the two *active* branches trigger on disjoint fitness ranges, one requiring $f_i \geq \delta_{\text{stable}}$ and the other $f_i < \delta_{\text{retire}}$, so at most one branch fires per skill per cycle. Eq. (9) is declarative; Algorithm 1 specifies the per-cycle ordering, with promotion and demotion preceding retirement and stabilization preceding mutation.

Table 6: Forging hyperparameters for SKILLFORGE. Slash-separated values apply to ALFWorld, WebShop, and Search-Augmented QA respectively; $S_{\max}$ and M scale with the initial library size of each environment.

Hyperparameter	Symbol	Value
Proto-fitness min usage	$N_{\min}$	3
Pre-retirement threshold	δ_{pre}	0.3
Preliminary eval episodes	M	500 / 450 / 400
Fitness warm-up threshold	N_{warm}	5
Default fitness (new skills)	f_{default}	0.5
Promotion min usage	N_{promote}	10
Retirement fitness threshold	δ_{retire}	0.4
Retirement min usage	N_{retire}	20
Stabilization fitness threshold	δ_{stable}	0.7
Stabilization min usage	N_{stable}	30
Demotion fitness threshold	δ_{demote}	0.5
Mutation fitness range	$[\delta_{\text{low}}, \delta_{\text{high}}]$	[0.4, 0.7]
Mutation min usage	N_{mutate}	5
Max mutations per cycle	K_{mutate}	5
Max retirements per cycle	K_{retire}	3
Total skill cap	$S_{\max}$	100 / 95 / 85
Original skill protection	N_{protect}	50
Forging frequency	E	2 (every 10 training steps)
Teacher model	$\mathcal{M}_T$	Kimi-K2.5 [Team et al., 2026]
Skill retrieval	top- K	6 (task-type matching)

Mutation weighting. We use the simplest monotone-decreasing weighting, $P(s_i) \propto 1 - f_i$, rather than sharper alternatives such as $1/f_i$ or $\exp(-\lambda f_i)$. Because mutation candidates already satisfy $\delta_{\text{low}} \leq f_i \leq \delta_{\text{high}}$ (default [0.4, 0.7]), the weights $1 - f_i$ lie in [0.3, 0.6], giving at most a $2\times$ preference for lower-fitness parents. This is enough to bias selection toward skills with more room to improve without concentrating mutation on a single outlier.

Threshold sensitivity. Table 7 sweeps the two most load-bearing thresholds, the retirement threshold δ_{retire} and the library cap $S_{\max}$, around their default values on ALFWorld. The overall success rate varies within $\pm 1.4\%$ across the swept range, showing robustness to threshold choices. We observe a mild concavity: overly aggressive retirement ($\delta_{\text{retire}} = 0.5$) removes borderline-but-useful skills before they stabilize, while overly permissive retirement ($\delta_{\text{retire}} = 0.3$) retains low-fitness skills that dilute the retrieved context. The default $\delta_{\text{retire}} = 0.4$ sits near the peak of this trade-off.

Table 7: Sensitivity of overall ALFWorld success (%) to the two most load-bearing thresholds, sweeping one parameter while fixing all others at their default values. Default values are boxed.

<i>Retirement threshold δ_{retire} (default 0.4)</i>				
δ_{retire}	0.3	0.4	0.5	0.6
Success (%)	91.3	92.4	91.8	91.0
<i>Library cap $S_{\max}$ (default 100)</i>				
$S_{\max}$	70	85	100	130
Success (%)	91.0	91.9	92.4	91.6

Compute resources. All RL runs use $64\times$ NVIDIA H200 GPUs under the verl framework. Estimated wall-clock per 200-step training run is ≈ 24 hours for ALFWorld, ≈ 18 hours for WebShop, and ≈ 72 hours for Search-Augmented QA (retrieval-dominated), for a total of ≈ 114 wall-clock hours $\times$ 64 GPUs = 7,296 GPU-hours across the three environments per full training run. The pre-retirement evaluation phase (Section 4.1) adds ≈ 6 hours per environment on the same hardware. Evaluation is run five times per reported number (sampling seeds {30, 40, 42, 50, 62}) and adds ≈ 1.5 – 4.5 hours per environment per seed. Teacher model overhead is negligible relative to RL: a forging cycle runs every $E = 2$ validation epochs (every 10 training steps) and issues up to $K_{\text{mutate}} = 5$ mutation calls per

cycle, for at most $\lceil 200/10 \rceil \times 5 = 100$ mutation calls per training run, all dispatched asynchronously during validation so they do not block GPU training. The teacher $\mathcal{M}_T$ is treated as a frozen black-box LLM: any sufficiently capable instruction-tuned model (proprietary or open-weight) can serve in this role, since the interface is only a structured template $\mathcal{P}_\mu$ and a single-shot text response. We use Kimi-K2.5 here for mutation quality, but the framework imposes no dependence on a specific teacher.

Mutation template. The mutation operator (Section 4.3, Eq. (10)) assembles the parent skill’s metadata, fitness statistics, and recent failure examples into the following structured template $\mathcal{P}_\mu$, which is sent to the teacher model $\mathcal{M}_T$:

Mutation Template $\mathcal{P}_\mu$

```
You are improving a skill for an embodied AI agent in {environment}.

The following skill has moderate performance (fitness={f_i}, used {U_i} times,
succeeded {C_i} times):

Original skill:
- ID: {skill_id}
- Title: {title}
- Principle: {principle}
- When to apply: {when_to_apply}

[If recent failures exist:]
This skill was involved in the following failed episodes:
- Task: {task description}
  Agent action: {failed action}
- ...

Please analyze these failure patterns and generate an improved skill that
specifically addresses them.

Generate ONE improved variant that addresses the weakness shown by its
moderate success rate. The improved skill should be more specific,
actionable, or correct than the original.

Return ONLY a JSON object with these fields:
{"skill_id": "{new_id}", "title": "...", "principle": "...",
"when_to_apply": "..."}

```

The template includes up to 8 recent failure examples (task description and agent action, each truncated to 200 characters) from episodes where the parent skill was retrieved but the episode failed. Failure examples are omitted when no failures are recorded. The teacher model returns a single JSON object, which is parsed and assigned a new skill identifier.

C SKILLFURNACE Dataset Details

SKILLFURNACE is a three-part dataset released alongside SKILLFORGE. Each part draws from a distinct data source within our experimental pipeline: (i) **SFT trajectories** come from rollouts of the Qwen2.5-7B-Instruct base model during the preliminary evaluation phase, before any RL training has begun; (ii) **evolved skill libraries** are initialized from the seed library released by SkillRL [Xia et al., 2026] and then logged at every forging cycle as one record per skill per snapshot; (iii) **retirement events** are lifecycle-gate decisions aggregated across the primary training run and additional previously recorded training runs (ablation variants and replicate seeds), so that the released event count is not limited to a single seed. Every mined event is labelled and quality-controlled under the same criteria as the primary run. In total, SKILLFURNACE contains **5,852 annotated records** across the three environments: 1,013 SFT trajectories, 4,521 library snapshot records, and 318 retirement events, summarized in Table 1. Table 8 summarizes the library component across environments, Table 1 reports dataset scale and retirement-event statistics, and Table 9 breaks retirement events down by failure category per environment.

SFT trajectories. The preliminary evaluation phase executes M rollout episodes with the Qwen2.5-7B-Instruct base model π_{θ_0} and retrieved skills injected via Eq. (2). Per-environment M values are 500 (ALFWorld), 450 (WebShop), and 400 (Search-Augmented QA); see Table 6 for all forging-

Table 8: SKILLFURNACE evolved-library composition per environment. Initial $|S_0|$ is the seed library released by SkillRL [Xia et al., 2026]; *total ever created* counts every skill ever introduced, including all mutation children; *retired during training* counts skills that reached `retired`; *final non-retired* is the active library size at the last training step. The final row shows what fraction of the surviving library originated from LLM-guided mutation.

Statistic	ALFWorld	WebShop	Search-Aug. QA	Total
Initial $ S_0 $	44	66	41	151
Total ever created	132	121	111	364
Retired during training	32	26	26	84
Final non-retired	100	95	85	280
of which <code>stable</code>	76	80	64	220
of which mutated ($g_i \geq 1$)	57	29	44	130

related hyperparameters. We retain every successful trajectory whose retrieved-skill set is disjoint from the pre-retired set $S_{\text{retire}}^{(0)}$, yielding the SFT corpus $\mathcal{D}_{\text{SFT}}$ defined in Eq. (4): 412 trajectories in ALFWorld, 348 in WebShop, and 253 in Search-Augmented QA, for a total of 1,013 released SFT trajectories. Each trajectory is stored as an ordered sequence of context–action pairs with the retrieved-skill ids attached to each context, so downstream practitioners can reconstruct the base corpus and re-filter it under alternative retention rules without rerunning rollouts. All trajectories in this component come exclusively from the base Qwen2.5 model, making them a clean cold-start fine-tuning resource decoupled from any specific RL setup.

Evolved skill libraries. Every forging cycle emits a library snapshot recording, for each skill, the id, name, behavioral principle b , applicability condition ω , cumulative usage count U_i , cumulative success count C_i , current fitness f_i , lifecycle state $\ell(s_i) \in \{\text{trial}, \text{active}, \text{stable}, \text{retired}\}$, generation depth g_i , and parent id for mutated skills. The initial library S_0 logged at snapshot 0 is the seed library released by SkillRL [Xia et al., 2026]; subsequent snapshots track how this seed evolves under RL. Forging runs every 10 training steps over the 200-step budget, producing 21 snapshots per run. Each snapshot emits one record per skill tracked at that step, yielding 1,512 records in ALFWorld, 1,683 in WebShop, and 1,326 in Search-Augmented QA, for a total of 4,521 library snapshot records. Table 8 reports the composition at the last training step. Across the three environments, the library grows from 44, 66, and 41 seed skills to a final non-retired size of 100, 95, and 85 at the cap S_{max} , with 132, 121, and 111 skills created cumulatively. Of the final non-retired skills, 76, 80, and 64 reach the `stable` state, and 57, 29, and 44 originate from LLM-guided mutation ($g_i \geq 1$), making 46.4% of final skills across the three environments children rather than seed skills. Snapshots are released together with per-cycle forging logs, so any fitness trajectory or lifecycle transition can be reconstructed offline.

Retirement events. Each retirement event records the full autopsy of a retired skill: the skill content, the complete fitness history leading up to retirement, the lifecycle-transition log, parent and sibling skills from the mutation tree, a human-annotated failure category drawn from the 8-way taxonomy described in Appendix E.2, and a free-text *key flaw* field describing the retired skill’s failure mode. To make the released sample size independent of any single seed or ablation setup, events are aggregated from the primary training run and additional previously recorded training runs; the per-environment totals in Table 1 therefore exceed the primary-run retirement counts reported in Section 5.4 by design. Retirement events exhibit an average peak fitness of 0.67 and an average lifespan of 38.0 training steps, with fitness dropping by 0.28 on average at retirement; 57.2% first reached the `stable` state before this decay, the defining pattern of *delayed obsolescence* that motivates the forging lifecycle. Figure 5 shows the lifespan distribution: retirements spread across the full training horizon rather than clustering at any single stage, so the retirement gate fires throughout RL rather than only at initialization or endgame. Table 9 reports the per-environment breakdown across the eight categories. Overly rigid ordering is the dominant mode in every environment (74.6% / 77.7% / 67.8%), confirming that procedural rigidity rather than content omission is the principal failure mode of LLM-distilled skills. Environment-specific patterns also emerge: *hallucinated action* accounts for 10.7% of WebShop events, corresponding to skills that reference UI elements absent from the WebShop environment, and *contradicts environment* is exclusive to Search-Augmented QA, corresponding to skills that invoke search operations the environment does not expose.

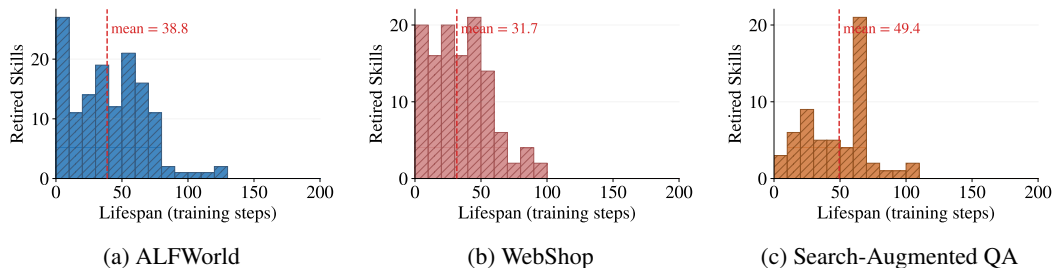


Figure 5: **Lifespan distribution of retirement events in SKILLFURNACE**, measured as `retired_step - born_step`. Retirements spread across the full training horizon rather than clustering at any single stage; the red dashed line marks the per-environment mean.

Table 9: SKILLFURNACE retirement events by failure category (8-way human-annotated taxonomy). Percentages are within environment. *Overly rigid ordering* dominates in every environment, confirming that procedural rigidity, not content omission, is the principal failure mode of LLM-distilled skills. Environment-specific patterns also emerge: *hallucinated action* appears mostly in WebShop (UI elements absent from the environment) and *contradicts environment* appears only in Search-Augmented QA (search operations absent from the environment).

Failure category	ALFWorld	WebShop	Search-Aug. QA	Total
Overly rigid ordering	103 (74.6%)	94 (77.7%)	40 (67.8%)	237 (74.5%)
Overly specific	26 (18.8%)	2 (1.7%)	2 (3.4%)	30 (9.4%)
Premature specialization	6 (4.3%)	10 (8.3%)	0 (0.0%)	16 (5.0%)
Hallucinated action	0 (0.0%)	13 (10.7%)	0 (0.0%)	13 (4.1%)
Redundant with policy	2 (1.4%)	1 (0.8%)	7 (11.9%)	10 (3.1%)
Contradicts environment	0 (0.0%)	0 (0.0%)	7 (11.9%)	7 (2.2%)
Scope mismatch	0 (0.0%)	0 (0.0%)	3 (5.1%)	3 (0.9%)
Vague or tautological	1 (0.7%)	1 (0.8%)	0 (0.0%)	2 (0.6%)
Total	138	121	59	318

549 **Release format.** SKILLFURNACE is released as three top-level JSON files
550 (`sft_trajectories.json`, `library_snapshots.json`, `retirement_events.json`) with
551 a schema document. All records are keyed by a global `skill_id` so the three components can be
552 joined: every skill id appearing in `retirement_events.json` is also present in the matching run’s
553 `library_snapshots.json` for every step between birth and retirement, and every skill retrieved in
554 `sft_trajectories.json` is resolvable in the step-0 library snapshot. The dataset is released under
555 CC-BY-4.0.

556 D Search-Augmented QA Full Per-Dataset Results

557 Table 10 reports accuracy on each of the seven Search-Augmented QA sub-datasets that are sum-
558 marised by the radar chart in Figure 2a. Both in-domain datasets ($\dagger$: NQ, HotpotQA) and out-of-
559 domain datasets ($\star$: TriviaQA, PopQA, 2WikiMultiHopQA, MuSiQue, Bamboogle) are included.
560 The 48.7% aggregate number reported in the main text corresponds to micro-averaged accuracy over
561 the 51,713-sample concatenation of these seven test sets.

562 SKILLFORGE obtains the best or second-best accuracy on every sub-dataset, with the largest absolute
563 improvements on the multi-hop benchmarks Bamboogle (+3.4 over the strongest baseline) and
564 HotpotQA (+0.7). Gains are consistent on both in-domain and out-of-domain splits, indicating that
565 the forging lifecycle does not overfit to the training distribution.

Table 10: Per-dataset accuracy on Search-Augmented QA (%). $\dagger$: in-domain; *: out-of-domain. Per-dataset cells for non-SKILLFORGE rows are taken from their respective source papers. The **Overall** column is recomputed as the sample-weighted micro-average over the 51,713-sample union shown in *Test samples*, so that all methods are compared on an identical test set; these values may differ from source-paper totals that used different evaluation configurations. **Bold**: best; underline: second best. “–”: not reported.

Method	NQ $\dagger$	TriviaQA*	PopQA*	HotpotQA $\dagger$	2Wiki*	MuSiQue*	Bamboogle*	Overall
<i>Test samples</i>	3,610	11,313	14,267	7,405	12,576	2,417	125	51,713
<i>In-context / retrieval baselines</i>								
Vanilla [Team, 2023]	11.6	35.6	1.2	16.4	22.2	4.8	14.4	16.9
CoT	12.8	35.6	3.8	16.2	22.6	6.6	24.0	17.9
RAG	27.4	58.2	17.8	25.8	23.2	9.4	16.8	29.4
Search-o1 [Li et al., 2025]	19.4	40.6	11.4	17.0	27.0	8.6	30.4	22.9
R1-Instruct [Team, 2025a]	21.0	44.9	17.1	20.8	27.5	6.0	19.2	26.0
<i>RL-based search methods</i>								
Search-R1 [Jin et al., 2025]	39.3	61.0	39.7	37.0	40.1	14.6	36.8	42.9
ZeroSearch [Sun et al., 2025]	43.6	61.8	51.5	34.6	35.2	18.4	27.8	45.2
StepSearch [Zheng et al., 2025]	–	–	–	38.6	36.6	22.6	40.0	–
EvolveR [Wu et al., 2025]	43.5	<u>63.4</u>	44.6	38.2	42.0	15.6	54.4	45.8
SkillRL [Xia et al., 2026]	<u>45.9</u>	63.3	45.9	<u>43.2</u>	40.3	20.2	<u>73.8</u>	<u>46.8</u>
SKILLFORGE (Ours)	48.2	65.0	<u>50.1</u>	43.9	<u>40.4</u>	<u>20.3</u>	77.2	48.7

E Case Studies

E.1 Deepest Forging Lineages

As shown in Figure 6, mutation lineages exhibit a heavy-tailed depth distribution: most refinements converge to a stable variant within one or two generations, while a tail extends up to nine generations when niches remain. Useful variants stabilize and stop mutating, so only lineages with room to specialize keep branching.

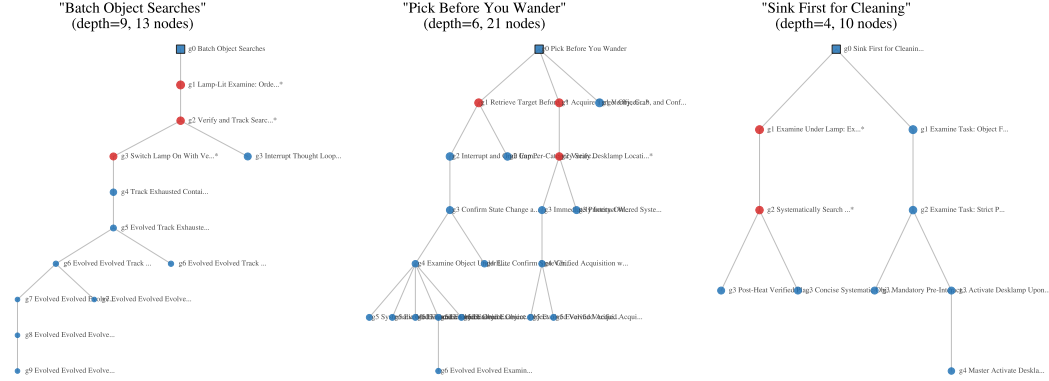


Figure 6: **Deepest forging lineages** on ALFWorld. Each subplot shows a root skill (gen=0, square) and its multi-generation descendants (circles). Red = retired, blue = stable. Early variants are often retired while later, more refined variants survive.

E.2 Failure Taxonomy Details

As shown in Figure 7, the 318 retirement events were labelled into one of eight failure categories. Two annotators labelled the events under this 8-way taxonomy; an initial pass reached Cohen’s $\kappa = 0.5$ inter-annotator agreement, and remaining disagreements were resolved by discussion. The dominant failure mode is *overly rigid ordering* at 74.5%, where the skill imposes a fixed procedural sequence that fails for edge cases; the second most common is *overly specific* at 9.4%, where skills work for narrow task subsets but fail broadly. Only 10 of the 318 retirement events (3.1%) are labeled *redundant with policy*; the remaining 96.9% contain prescriptions inconsistent with successful rollouts—rigid ordering, hallucinated operations, overfit specializations, self-contradictions—so *delayed obsolescence* reflects once-stable skills becoming concretely misleading, not the policy

582 outgrowing harmless crutches. Environment-specific patterns also emerge: *contradicts environment*
583 failures appear exclusively in Search-Augmented QA, where LLM-guided mutation hallucinates
584 search operations absent from the environment.

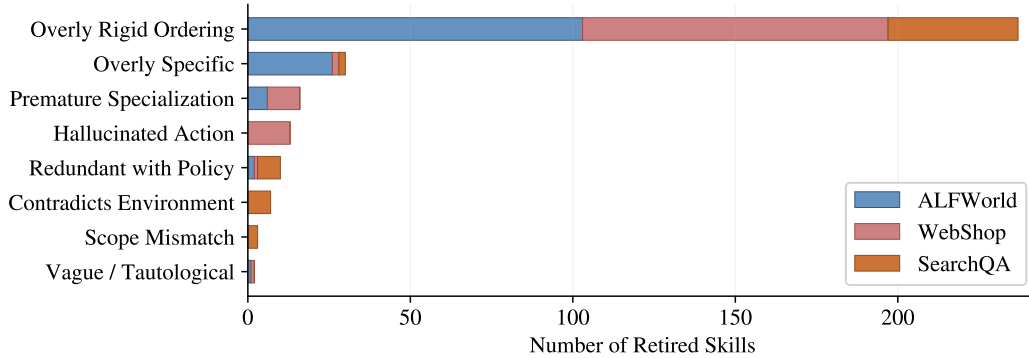


Figure 7: Failure taxonomy of 318 retirement events across three environments. Blue: ALFWorld; red: WebShop; orange: Search-Augmented QA. Overly rigid ordering dominates across all environments; environment-specific failure modes emerge in Search-Augmented QA.

585 E.3 WebShop: Hidden-Constraint Verification

586 **Parent skill** (“Confirm Hidden Specs”, fitness=0.84, gen=0):
587 *Principle*: Scroll to bullet points/description to verify non-title attributes (material, care, fit,
588 design features); reject items that satisfy title keywords but fail hidden constraints.
589 *When to apply*: When evaluating candidate products against multi-attribute instructions.

590 **Successful child** (“Systematically Verify All Hidden Constraints”, fitness=0.70, gen=1,
591 stable):
592 *Principle*: After selecting a candidate product, explicitly open the product descrip-
593 tion/features section and verify every constraint from the instruction that is not visible
594 in the title or options.
595 *When to apply*: When a candidate product matches title keywords but contains hidden
596 attribute constraints.

597 **Failed child** (“Systematically Verify Every Non-Visible Constraint”, fitness=0.21, gen=1,
598 retired at step 78):
599 *Principle*: After selecting a product, scroll down and click on the ‘Description’ or ‘Features’
600 tab/section to read every specification.
601 *When to apply*: Whenever a product page is opened.

602 The successful child reaches *stable* status by maintaining high accuracy within its target niche; its
603 aggregate fitness of 0.70 is below the parent’s 0.84 because the child targets a narrower set of tasks,
604 but within-niche accuracy is what matters for the lifecycle, not aggregate fitness. The failed variant
605 referenced interface elements such as “click on the Description tab,” which is absent from WebShop;
606 its fitness decayed rapidly and the retirement gate filtered it out at step 78. The trial gate filters the
607 hallucinated-interface variant before it affects the active library.

608 E.4 ALFWorld: Mutation Outcomes

609 The initial skill “Pick Before You Wander” produced two mutations with opposite outcomes:

610 **Parent** (“Pick Before You Wander”, gen=0, fitness=0.84):
611 *If the target object is visible or discovered, take it immediately; never leave it behind to*
612 *explore other places.*

613

614 **Successful child** (“Verify, Grab, and Confirm Pickup”, gen=1, fitness=0.70, stable):
615 *When a needed object is visible, move directly to it, take it immediately, and then verify it is*
616 *now in your inventory before proceeding.*
617 → Adds a **verification step** that catches pickup failures.

618

619 **Failed child** (“Retrieve Target Before Locating Desklamp”, gen=1, fitness=0.24, **retired**
 620 at step 65):

621 *First locate and pick up the target object specified in the goal before navigating to any*
 622 *furniture.*

623 → Introduces a rigid ordering that fails for tasks where the lamp must be found first.

624 From the same parent, mutation produced both a beneficial specialization (adding a verification
 625 step) and a harmful over-specification (imposing a rigid task ordering). The retirement mechanism
 626 correctly identified and removed the harmful variant.

627 E.5 Search-Augmented QA: Query Reformulation Lineage

628 A multi-generation forging chain on Search-Augmented QA:

629 **Gen-0:** “Disambiguation Add-Ons” (fitness=0.88)

630 *If the name is ambiguous, append clarifiers (birth year, nationality, known work) to the*
 631 *query.*

632 ↓ mutation

633 **Gen-1:** “Structured Query Escalation with Fallback” (fitness=0.21, **retired** at step 83)

634 *Systematically escalate through three reformulation strategies when initial results lack*
 635 *evidence.*

636 → Too rigid; forced 3-step escalation even for simple queries.

637 ↓ later mutation from same lineage

638 **Gen-1:** “Verify Before Acting” (fitness=0.70, **stable**)

639 *Before performing any action on an entity, first confirm its identity by searching with*
 640 *disambiguating context.*

641 → Simpler, more general, and more effective.

642 Forging explores multiple directions from the same parent: the overly complex variant is retired while
 643 the simpler, more robust variant survives.

644 F KL Regularization under a Library-Conditioned Context

645 This appendix elaborates on how the KL penalty in Eq. (1) composes with the evolving skill library,
 646 complementing the non-stationarity of the fitness estimator formalized in Eq. (11).

647 Under our formulation (Eq. (2)), the library $\mathcal{S}^{(k)}$ enters the policy through the context p_t rather than
 648 through the policy parameters θ . The parameterization of π_θ is therefore invariant across forging
 649 cycles; what varies over training is the prompt distribution fed to the fixed network. The KL term
 650 $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ is always evaluated at the same context $c_{n,l}$ seen by both π_θ and π_{ref} , so the divergence
 651 is well-defined at every training step, regardless of how far $\mathcal{S}^{(k)}$ has drifted from $\mathcal{S}^{(0)}$.

652 Two structural properties align the fixed anchor π_{ref} with the library-conditioned inputs it encounters
 653 during RL. First, mutated skills are teacher-model rewrites of surviving parents that preserve the
 654 (v, b, ω) schema of Section 3.2, so new prompts differ from SFT-time prompts mainly in skill content
 655 rather than in structural form, on which π_{ref} remains a calibrated next-token predictor. Second, we
 656 use $\beta = 10^{-3}$ (Table 5), so the KL term acts as a soft regularizer against large single-step deviations
 657 rather than a tight anchor to the SFT policy’s full conditional distribution. An alternative design is to
 658 re-anchor π_{ref} at each forging cycle, trading the clean separation between SFT initialization and RL
 659 optimization for tighter alignment between the reference policy and the current library; this variant
 660 is compatible with the rest of our framework, and the fixed-anchor choice is the simpler default we
 661 adopt here.

662 G Limitations

663 Fitness is defined at the episode level: the K retrieved skills share a single binary reward (Eq. (6)).
 664 This is an approximation—in episodes where multiple skills are co-retrieved, the signal cannot
 665 distinguish individual skill contributions. We mitigate this via applicability-conditioned retrieval,

which spreads skills across task types; the top co-retrieval frequency across all skill pairs remains under 15% of episodes in each environment, so in practice each skill’s fitness estimate is dominated by episodes where it was retrieved alone. Nevertheless, the shared-credit assumption may slow convergence on tasks where skill co-retrieval is structurally unavoidable.

All experiments use a 7B base model; lifecycle dynamics under larger models may differ, and we leave scalability to future work. LLM-guided mutation relies on an external teacher model $\mathcal{M}_T$: performance is contingent on the teacher’s instruction-following ability. Any sufficiently capable instruction-tuned model can serve this role (Appendix B uses Kimi-K2.5 but imposes no dependence on a specific provider), but a weak teacher will produce low-quality mutations that the trial gate must filter.

H Discussion

Design choices. Fitness is tracked as a cumulative running average (Eq. (6)), which integrates evidence over the full lifecycle of each skill. This is the natural estimator for our detection target, skills whose accumulated success record needs to be distinguished from skills that were never effective; it is what makes “once stable, now failing” a well-defined pattern to catch. A faster estimator such as an exponential moving average would respond more quickly to fitness changes, but this sensitivity cuts both ways: a skill that encounters a transiently hard batch of tasks would be flagged before the policy has stabilized, producing false retirements. The cumulative average’s inertia is intentional—the lifecycle’s usage-count gates (N_{retire} , N_{stable}) already guard against premature decisions; within that framework, the cumulative estimate reliably separates skills with a strong historical record from those that have consistently underperformed. Table 1 reports an average retirement-event lifespan of 38.0 training steps, showing that retirement occurs promptly relative to the 200-step training budget despite the conservative estimator.

Mutation survival. Across the three environments, 213 mutant skills were generated: 88 in ALFWorld, 55 in WebShop, and 70 in Search-Augmented QA. The majority survive the lifecycle: the mutant survival rate is 64.8%, 52.7%, and 62.9% respectively, so LLM-guided mutation produces viable skills more often than not. Each forging cycle requires up to K_{mutate} teacher model calls for mutation plus lightweight fitness bookkeeping.

Ablation library dynamics. In Table 3 the –mutation and –forging variants both retain the initial library size (44 on ALFWorld, 66 on WebShop, 41 on Search-Augmented QA). These numbers refer to $|S_0|$, the distilled library *before* pre-retirement, to make cross-variant comparison direct. For –forging the entire forging method, including pre-retirement, runtime retirement, and mutation, is disabled, so the SFT data is unfiltered and no skills are added or removed after initialization. For –mutation, pre-retirement and runtime retirement both run, but the two mechanisms remove very few skills on these environments in practice: pre-retirement at $\delta_{\text{pre}} = 0.3$ with $N_{\text{min}} = 3$ discards only initial skills that demonstrably hurt the base-model policy, and on the three environments this is a small fraction of S_0 (typically 0–3 skills); the remaining skills then accumulate enough usage during RL to stay above δ_{retire} , so runtime retirement triggers rarely when mutation is off and no low-fitness children are being introduced. Size changes driven by the lifecycle therefore manifest primarily once mutation is re-enabled, which is consistent with the joint role of mutation and retirement reported in Section 5.3. The –pre-retirement variant confirms this picture from the opposite direction: with runtime retirement and mutation intact, the final library still settles at the cap (100, 95, 85) because the cap is the binding constraint once mutation is active. The 1.2–3.5% success drop therefore arises not from library size but from SFT data quality: pre-retirement is an integral first stage of the lifecycle that shapes the initial policy $\pi_{\theta_{\text{st}}}$ on which the subsequent retirement and mutation operate.

The –retirement column in Table 3 reaches 132, 121, and 111 skills on the three environments, matching the *total-ever-created* count of the full method. This follows from the forging schedule: the number of forging cycles and the per-cycle budget K_{mutate} are fixed, so the cumulative number of child skills generated is largely invariant to whether low-fitness parents are subsequently removed. Retirement determines which skills remain in the active library at any point, not how many are produced during training; correspondingly the full method’s active library settles at a much smaller size (100, 95, 85) while –retirement accumulates every generated skill.

718 **Evaluation protocol.** All reported numbers use stochastic decoding at temperature 1.0 for both
719 training-time rollouts and test-time evaluation, so that evaluation noise reflects the policy’s deployed
720 behavior rather than a degenerate argmax. Each reported score is averaged over five evaluation runs
721 of the best validation checkpoint with sampling seeds {30, 40, 42, 50, 62}. The mean $\pm$ standard
722 deviation of the overall success rate is $92.4 \pm 0.5\%$ on ALFWorld, $78.4 \pm 0.4\%$ on WebShop, and
723 $48.7 \pm 0.3\%$ on Search-Augmented QA, with the low variance confirming that reported results are
724 stable under resampling.

725 **Statistical significance.** We test whether SKILLFORGE’s overall success rate exceeds the strongest
726 memory-augmented RL baseline, SkillRL, using a one-sample t -test against the SkillRL point
727 estimate under the null $H_0 : \mu_{\text{SKILLFORGE}} = \mu_{\text{SkillRL}}$ with our five-seed evaluation sample ($n = 5$,
728 $\text{df} = 4$). On ALFWorld ($t = 11.18$, $p = 1.8 \times 10^{-4}$), WebShop ($t = 31.86$, $p = 2.9 \times 10^{-6}$), and
729 Search-Augmented QA ($t = 14.16$, $p = 7.2 \times 10^{-5}$), the improvement is significant at $p < 10^{-3}$
730 on all three environments. The corresponding two-sided 95% confidence intervals, [91.78, 93.02],
731 [77.90, 78.90], and [48.33, 49.07], do not overlap the SkillRL baseline values of 89.9, 72.7, and
732 46.8, so the 1.9–5.7% absolute gains are unlikely to be explained by sampling noise. Under the
733 conservative assumption that SkillRL’s per-seed variance matches our observed per-environment
734 standard deviations of 0.5%, 0.4%, and 0.3% respectively, a pooled Welch’s t -test yields $t = 7.91$
735 (ALFWorld), $t = 22.51$ (WebShop), and $t = 10.03$ (Search-Augmented QA), with $p < 10^{-4}$ on all
736 three environments, preserving the conclusion.

737 **Broader impact.** Treating experience accumulation as a quality-control problem improves agent
738 performance and yields interpretable artifacts: the SKILLFURNACE dataset enables follow-up research
739 on skill quality prediction and failure-aware generation. The contributed collection contains only
740 SFT trajectories, evolved skill libraries with fitness annotations, and retirement events with failure
741 annotations, and introduces no risks beyond those inherent to LLM agents that act in interactive
742 environments.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction (Section 1) state three contributions: fitness-driven lifecycle, retirement-aware training method with SkillFurnace dataset, and empirical results across three benchmarks. All are supported by experimental results in Section 5.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Limitations are discussed in Appendix G, covering coarse credit assignment, cumulative fitness averaging, and model scale.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include formal theoretical results or proofs. The method is empirically validated.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 5.1 specifies the base model, training hyperparameters, and environment details. Appendix B provides the complete set of forging hyperparameters. Algorithm 1 gives the full forging cycle procedure.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: We will provide the SkillFurnace dataset and code upon publication. An anonymized version is not provided at submission time to preserve anonymity.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5.1 describes environments, baselines, model, optimizer, learning rate, batch size, group size, skill library sizes, and GPU configuration. Appendix B lists all forging hyperparameters.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Standard deviations across five evaluation runs (sampling seeds at the best validation checkpoint) are reported in Appendix H (± 0.3 – 0.5% on the overall metric of each environment).

791 **8. Experiments compute resources**

792 Question: For each experiment, does the paper provide sufficient information on the com-
793 puter resources (type of compute workers, memory, time of execution) needed to reproduce
794 the experiments?

795 Answer: [Yes]

796 Justification: Section 5.1 and Appendix B specify compute resources (64× NVIDIA H200
797 GPUs, verl framework) and report the estimated wall-clock training cost and teacher-LLM
798 call budget.

799 **9. Code of ethics**

800 Question: Does the research conducted in the paper conform, in every respect, with the
801 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

802 Answer: [Yes]

803 Justification: The research uses publicly available benchmarks and does not involve human
804 subjects or sensitive data.

805 **10. Broader impacts**

806 Question: Does the paper discuss both potential positive societal impacts and negative
807 societal impacts of the work performed?

808 Answer: [Yes]

809 Justification: Appendix H includes a broader impact statement. We do not foresee negative
810 impacts beyond those inherent to LLM agent systems.

811 **11. Safeguards**

812 Question: Does the paper describe safeguards that have been put in place for responsible
813 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
814 image generators, or scraped datasets)?

815 Answer: [N/A]

816 Justification: The SkillFurnace dataset contains only textual skill descriptions, SFT trajec-
817 tories, and fitness metrics, posing no risk for misuse.

818 **12. Licenses for existing assets**

819 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
820 the paper, properly credited and are the license and terms of use explicitly mentioned and
821 properly respected?

822 Answer: [Yes]

823 Justification: All benchmarks (ALFWorld, WebShop, Search-R1) and the base model
824 (Qwen2.5-7B-Instruct) are properly cited. These are publicly available research assets.

825 **13. New assets**

826 Question: Are new assets introduced in the paper well documented and is the documentation
827 provided alongside the assets?

828 Answer: [Yes]

829 Justification: The SkillFurnace dataset is documented in Section C with statistics, schema
830 description, and failure taxonomy. Full documentation will be provided.

831 **14. Crowdsourcing and research with human subjects**

832 Question: For crowdsourcing experiments and research with human subjects, does the paper
833 include the full text of instructions given to participants and screenshots, if applicable, as
834 well as details about compensation (if any)?

835 Answer: [N/A]

836 Justification: This work does not involve crowdsourcing or human subjects.

837 **15. Institutional review board (IRB) approvals or equivalent for research with human**
838 **subjects**

839 Question: Does the paper describe potential risks incurred by study participants, whether
840 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
841 approvals (or an equivalent approval/review based on the requirements of your country or
842 institution) were obtained?

843 Answer: [N/A]

844 Justification: This work does not involve human subjects research.

845 **16. Declaration of LLM usage**

846 Question: Does the paper describe the usage of LLMs if it is an important, original, or
847 non-standard component of the core methods in this research? Note that if the LLM is used
848 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
849 scientific rigor, or originality of the research, declaration is not required.

850 Answer: [Yes]

851 Justification: The method uses an LLM teacher model for skill distillation and mutation
852 (Section 4.3, Appendix B). The SkillFurnace failure categories in Section C are assigned by
853 human annotators.